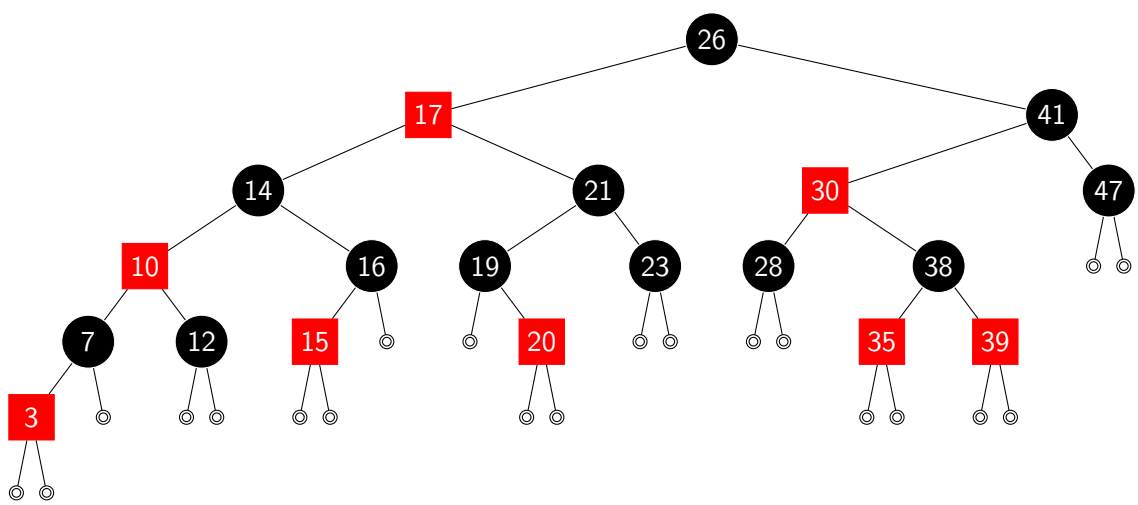
Introduction

During our investigation of the Binary Search Tree data structure we saw that the running time of most methods depending upon the height of the tree. Unfortunately, we could not meaningfully bound the height of the tree above (in the worst case the height was $\Theta(n)$). The question then becomes: can we modify our binary search tree data structure to ensure "reasonable" bounds on the height. This idea of a tree that "self-balances" is called a "self-balancing binary search tree"; we will examine **red-black** trees, but there are many different implementations that achieve this goal.

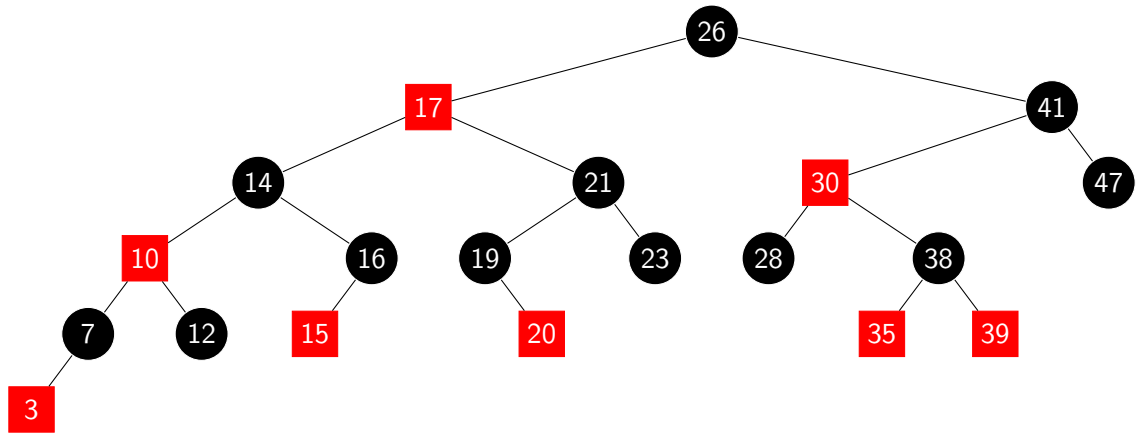
Definition 1. A **red-black tree** is a binary search tree with the following properties

1. Every node is either **red** or **black**.
2. The root is **black**.
3. Every leaf is NIL and is **black**.
4. If a node is **red**, then both its children are **black**.
5. For each node, all simple paths from the node to descendant leaves contain the same number of **black** nodes.

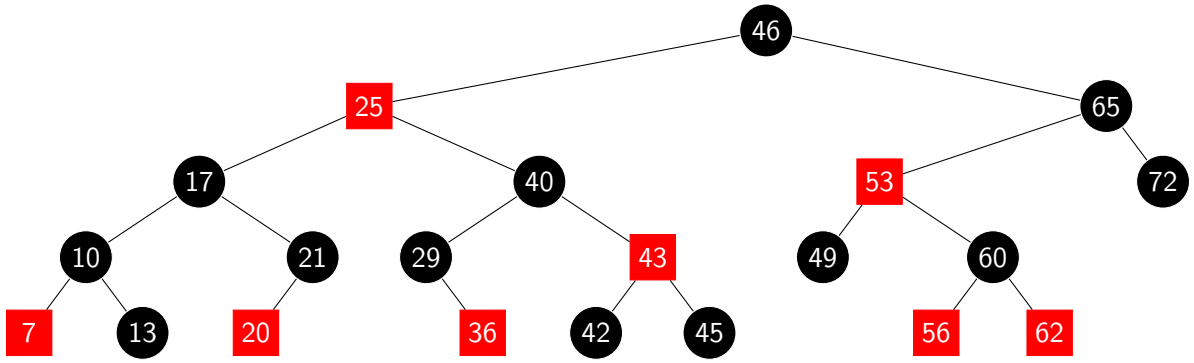
An example of a **red-black** tree can be found below; we leave it to the reader to verify that all of the properties are satisfied by this tree. The small circular nodes correspond to **NIL**.



Frequently, the **black NIL** leaves are not drawn explicitly since they are rather cumbersome. Omitting them causes no issue so long as we are careful when we examine "all paths leading from the root to a leaf" since interior nodes with a single non-**NIL** child still have a child that is a leaf node.

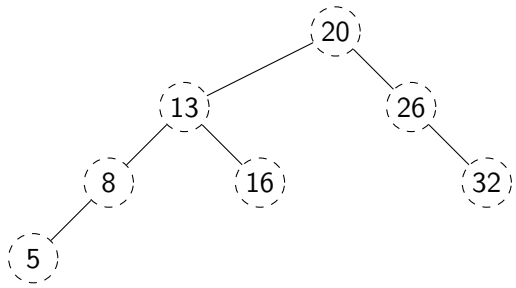


This tree is not a **red-black** tree. Why not?

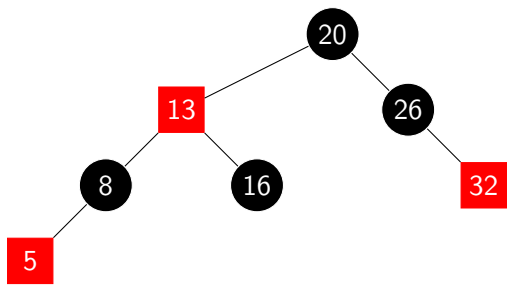


The path $(46, 25, 17, 10, 13, \textbf{NIL})$ contains 5 black nodes and the path $(46, 65, 72, \textbf{NIL})$ contains 4 black nodes, hence the tree is not a **red-black** tree.

Color the following tree so that it is a **red**-**black** tree



One valid coloring of this tree is



Let us verify that all of the properties are satisfied:

1. Every node is either **red** or **black**. ✓
2. The root is **black**. ✓
3. Every leaf is NIL and is **black**. ✓
4. If a node is **red**, then both its children are **black**.
   (a) Nodes 5, 13, and 32 are red and their children are all **black** nodes. ✓
5. For each node, all simple paths from the node to descendant leaves contain the same number of **black** nodes.
   (a) Path $(20, 13, 8, 5, \textbf{NIL})$ has 3 **black** nodes. ✓
   (b) Path $(20, 13, 8, \textbf{NIL})$ has 3 **black** nodes. ✓
   (c) Path $(20, 13, 16, \textbf{NIL})$ has 3 **black** nodes. ✓
   (d) Path $(20, 26, \textbf{NIL})$ has 3 **black** nodes. ✓
   (e) Path $(20, 26, 32, \textbf{NIL})$ has 3 **black** nodes. ✓

Theorems and correctness

Let h be the height of some **red-black** tree and x be a leaf of the **red-black** tree. How many nodes can be on a path from the root to x? This turns out to be a difficult question to answer immediately. Therefore, we shall analyze it via a sequence of theorems.

Theorem 1. A complete binary search tree of height h has $2^{h+1} - 1$ nodes.

Proof. The ith level has 2^i nodes, where $i = 0, 1, \ldots, h$. Therefore the total number of nodes is

$$n = 1 + 2 + 2^2 + 2^3 + \cdots + 2^h = 2^{h+1} - 1$$

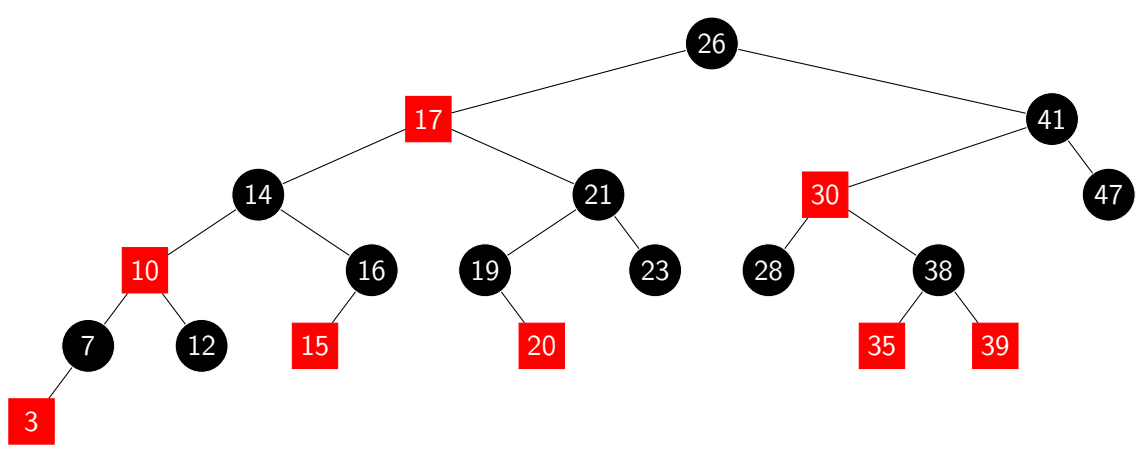
Lastly note that

$$\log_2(n+1) = 2^{h+1} \qquad \Longrightarrow \qquad h = \log_2(n+1) - 1$$

□

Theorem 2. A **red-black** tree of height h has at least $2^{\lceil h/2 \rceil} - 1$ internal nodes.

Proof. Let T be a **red-black** tree of height h. Let T' be the tree formed by removing all of the leaves from T. Note that the height of T' must be $h-1$. Let r be the root of T'. Since no child of a **red** node is **red** and r is **black**, the longest path from r to a leaf has at least $\lceil h/2 \rceil$ **black** nodes (see Lemma 1 at the end of the notes for more details). Since every path from r to a leaf has the same number of **black** nodes, every path from r to a leaf at least $\lceil h/2 \rceil$ **black** nodes. Thus, T' contains a complete binary tree of height at least $\lceil h/2 \rceil - 1$. By Theorem 1, tree T' has at least $2^{(\lceil h/2 \rceil - 1)+1} - 1$ nodes. Therefore, T has at least $2^{\lceil h/2 \rceil} - 1$ internal nodes. □



Notice that the tree has height 6 and there are 20 internal nodes; so $20 \geq 2^3 - 1 = 7$

Theorem 3 (**Red-black** tree height)**.** A **red-black** tree with n internal nodes has height at most $2\log_2(n+1)$,

Proof. Let T be a **red-black** tree with n nodes and height h. By Theorem 2, $n \geq 2^{\lceil h/2 \rceil} - 1$. Therefore,

$$\log_2(n+1) \geq \lceil h/2 \rceil \geq h/2 \qquad \Longrightarrow \qquad h \leq 2\log_2(n+1)$$

□

So,

$$\log_2(n+1) - 1 \leq h \leq 2\log_2(n+1)$$

therefore $h \in \Theta(\log(n))$.

Methods and analysis

We shall limit our discussion to insertion of nodes into a **red-black** tree; deletion is equally as cumbersome. Insertion begins by first locating the parent of the new node in the exact same manner as with a binary search tree. We then insert the node and color it red. Once we have colored the node, we simply "fix" the **red-black** tree properties.

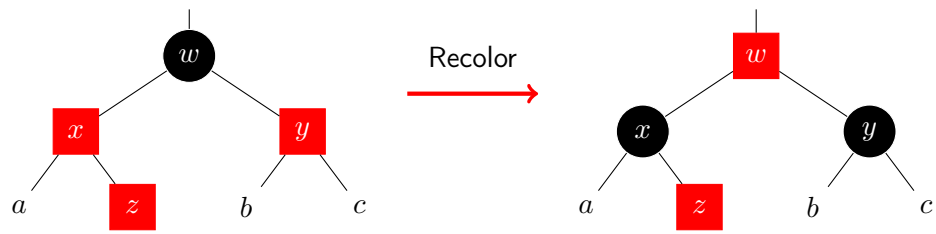
```
function RBLocateParent(T,z)
    ▷ Return future parent of z in tree T
    y = NIL
    x = T.root
    while x is not a leaf do
        y = x
        if z.key < x.key then
            x = x.left
        else
            x = x.right
        end if
    end while
    return y
end function
```

```
function RBTreeInsert(T,z)
    y = RBLocateParent(T, z)
    z.parent = y
    if y = NIL then
        T.root = z
    else if z.key < y.key then
        y.left = z
    else
        y.right = z
    end if
    z.left = leaf
    z.right = leaf
    z.color = red
    RBInsertFixup(T,z)
end function
```

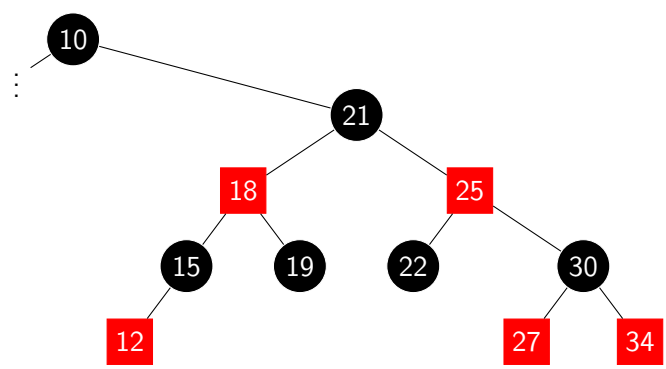
What are the possibilities that need fixing?

Case I If the parent and "auncle" (the sibling of the parent) of z are **red**

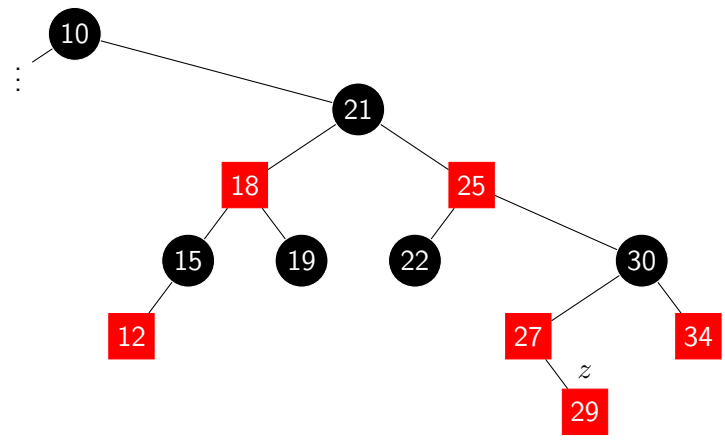
- Color the parent z **black**.
- Color the auncle of z **black**.
- Color the grandparent of z **red**.
- Repeat on the grandparent of z.



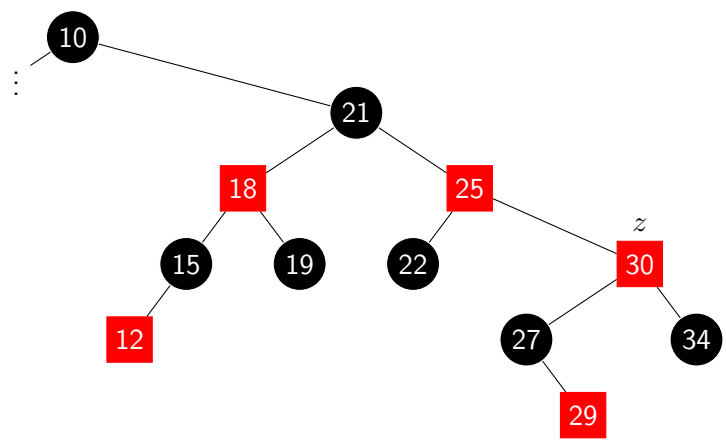
Perform `RBTreeInsert`(T,29) on the tree given below



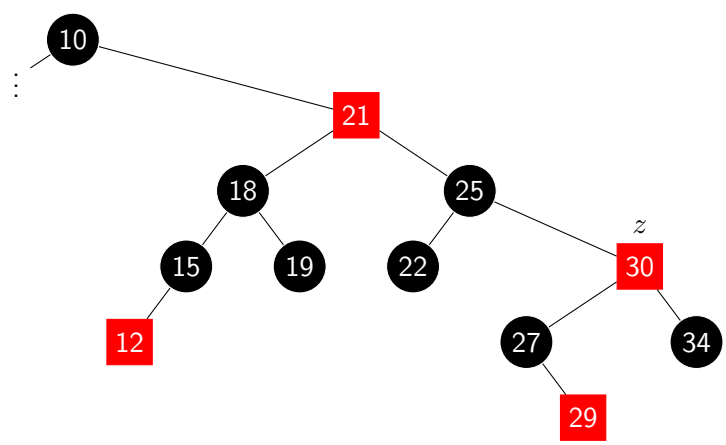
Insert the node



Since the parent, 27, and auncle, 34, of z are both red we recolor both 27 and 34 to be **black** and the grandparent of z, 30, to be red. Afterward z is assigned to 30.



Both parent and auncle are red again, so we repeat the process again.

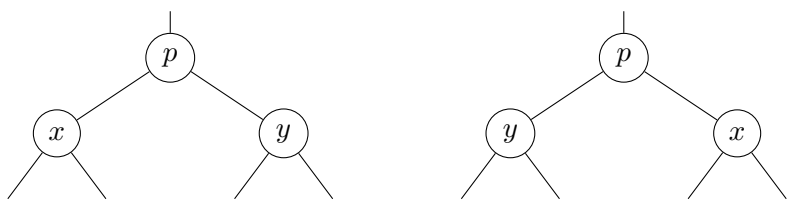


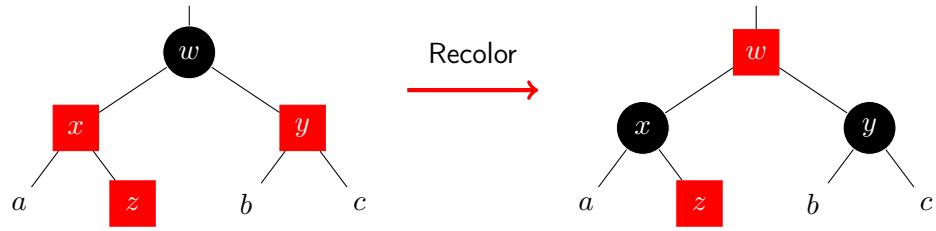
The tree is now a valid **red-black** tree.

This method requires finding the sibling of a node, so we abstract that notion into a separate function,

```
function Sibling(x)
    ▷ Returns the sibling of x
    if x.parent = NIL then
        error "Root has no siblings."
    end if
    p = x.parent
    if p.left = x then
        return p.right
    else
        return p.left
    end if
end function
```

You can visualize the code with the following diagram; y is always the sibling of x, we just need to check for the "other child" of the parent.






```
function RBInsertFixupA(T, alters z)
    while (z ≠ T.root) and (z.parent.color ≠ black) do
        y = Sibling(z.parent)
        if y.color = black then
            return
        end if
        z.parent.color = black
        y.color = black
        z = z.parent.parent
        z.color = red
    end while
end function
```

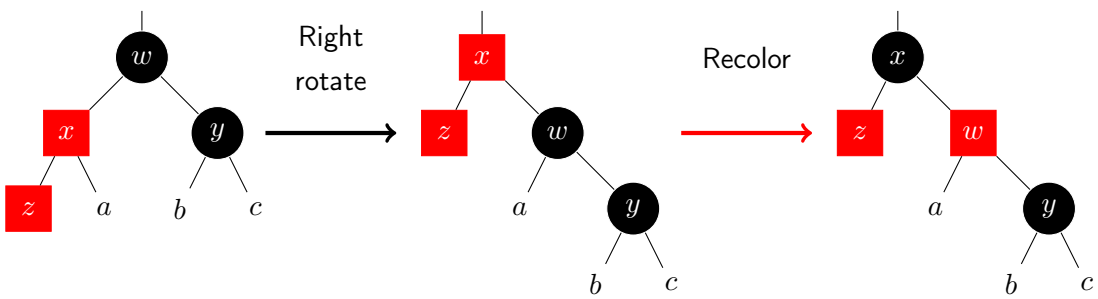
The code for implementing this is rather self-explanatory; we notice that we need to check whether z is the root of the tree in order to avoid trying to access member of **NIL** (which by definition has no member variables).

Furthermore, we observe that we always ascend the tree by two levels so the while loop runs at most $h/2$ times and each iteration has cost $\Theta(1)$, therefore the run time is in $O(h)$.

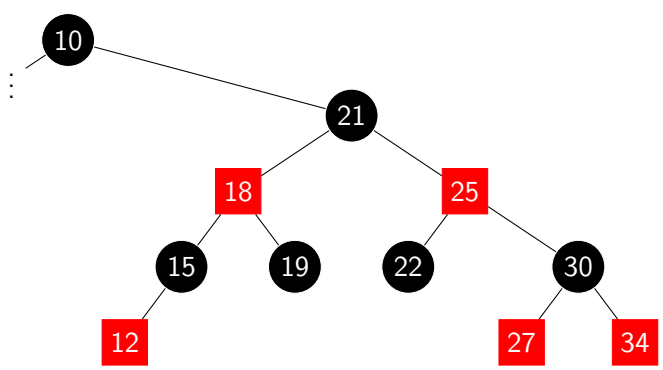
The next case we shall examine is inexplicably Case III; the reasoning for this will become apparent once we analyze Case II.

Case III If the parent of z is **red** and its auncle is **black**. If z is a left child and its parent is a left child,

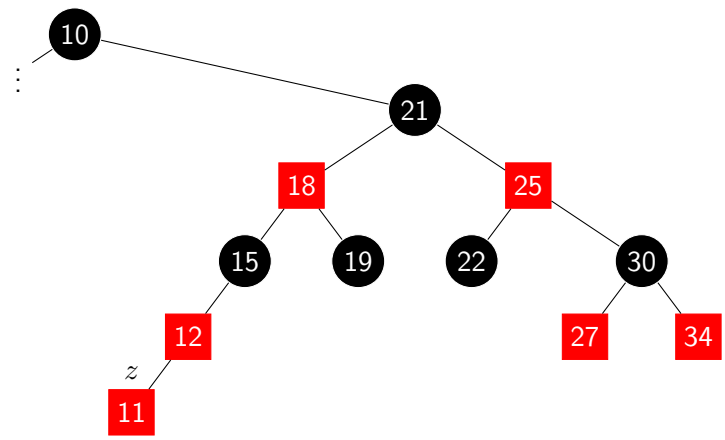
- Right Rotate on the grandparent of z,
- Color the parent of z **black**,
- Color the sibling of z **red**



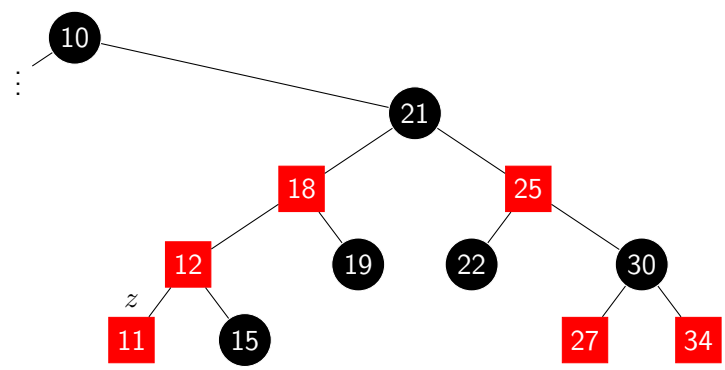
Perform `RBTreeInsert`(T,11) on the tree given below



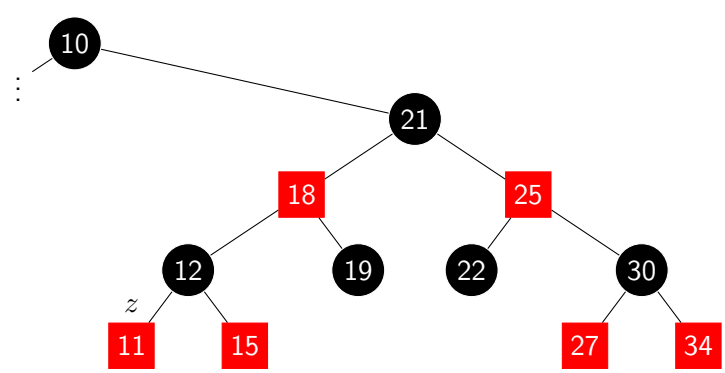
Insert the node

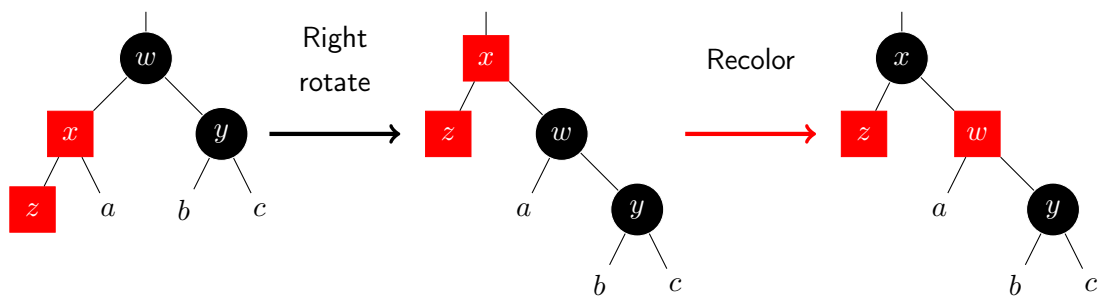


The parent, 12, of z is red, the auncle is **black**, z is a left child, and z.`parent` is also a left child. Therefore, we right rotate on 15,



Then recolor the parent of z to be **black**, and recolor the sibling of z to be **red**.





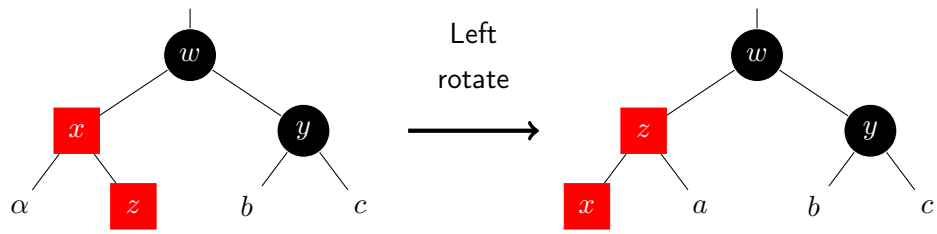
function `RBInsertFixupC`(T, **alters** z)
  if ($z = T.$`root`) **or** ($z.$`parent.color` $=$ **black**) **then**
    return
  end if
  $x = z.$`parent`
  $w = x.$`parent`
  if ($z = x.$`left`) **and** ($x = w.$`left`) **then**
    `RightRotate`(T,w)
    $x.$`color` $=$ **black**
    $w.$`color` $=$ **red**
  else if ($z = x.$`right`) **and** ($x = w.$`right`) **then**
    `LeftRotate`(T,w)
    $x.$`color` $=$ **black**
    $w.$`color` $=$ **red**
  end if
end function

The code here is again rather simple, we notice that this case can also be applied of z and its parent are both right children. In this case we mirror all of the operations and the same result is achieved.

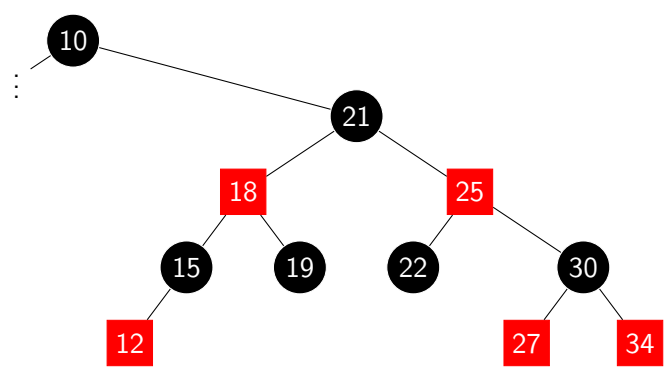
Every operation performed is pointer rearrangement which takes constant time, so the run time is in $\Theta(1)$.

Case II If the parent x of z is **red** and its auncle is **black**. If z is a right child and its parent x is a left child,

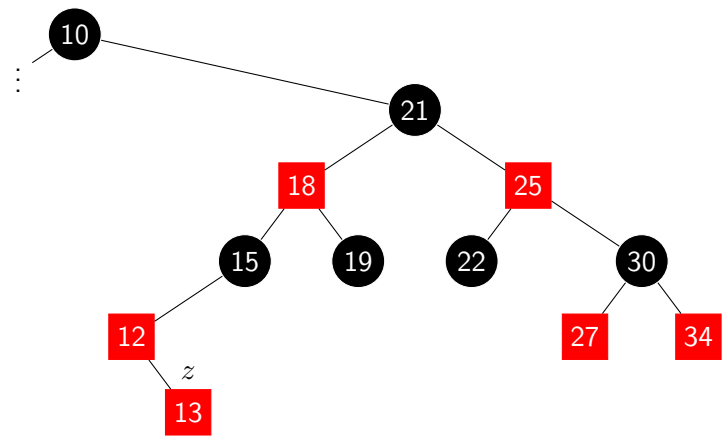
- Left Rotate on x,
- Apply algorithm from Case III on x



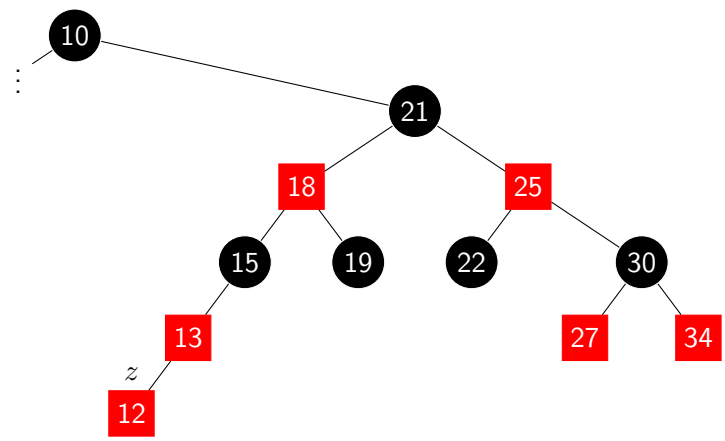
Perform `RBTreeInsert`(T,13) on the tree given below



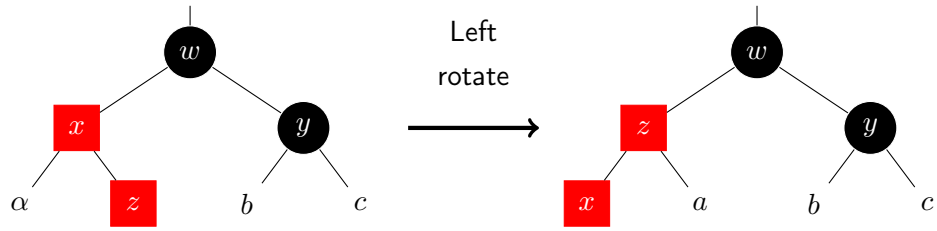
Insert the node



The parent, 12, of z is red, the auncle is **black**, z is a right child, and z.`parent` is a left child. Therefore, we left rotate on 12 and set z to be 12,



We are now in Case III and leave it to the reader to finish the analysis by applying it.

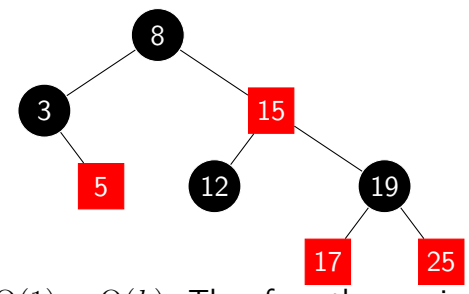


function `RBInsertFixupB`(T, **alters** z)
  if ($z = T$.`root`) **or** (z.`parent`.`color` = **black**) **then**
    return
  end if
  $x = z$.`parent`
  $w = x$.`parent`
  if ($z = x$.`right`) **and** ($x = w$.`left`) **then**
    $z = x$
    `LeftRotate`(T,x)
  else if ($z = x$.`left`) **and** ($x = w$.`right`) **then**
    $z = x$
    `RightRotate`(T,x)
  end if
end function

As we saw with Case III, the code here is very simple and can be applied to the opposite case as well (when z is a left child and z.`parent` is a right child).
Unsurprisingly, the run time of this method is in $\Theta(1)$.

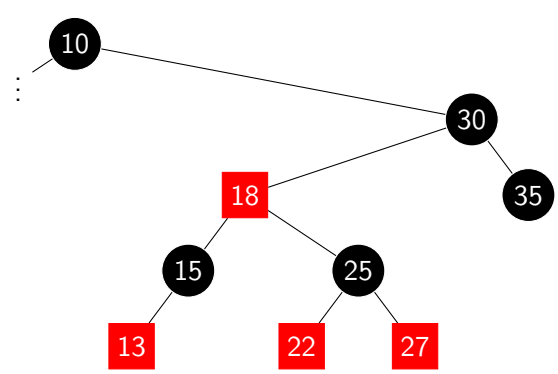
In order to fix the tree we may need to apply all three methods in order. We begin (potentially) with Case I where we "push the redness" of the tree; we then (potentially) perform some sequence of rotations and recoloring some of the nodes to ensure that the **red**-**black** tree properties are preserved. Finally, it is possible that we have recolored the root of the tree while fixing it; if this occurs then we must color it to be **black** again. Whenever we recolor the root to be **black** we have increased the total number of **black** nodes in the tree by 1. We shall not prove that this guarantees that the properties of the tree are preserved.

function `RBInsertFixup`(T,z)
  `RBInsertFixupA`(T,z)
  `RBInsertFixupB`(T,z)
  `RBInsertFixupC`(T,z)
  T.`root.color` = **black**
end function

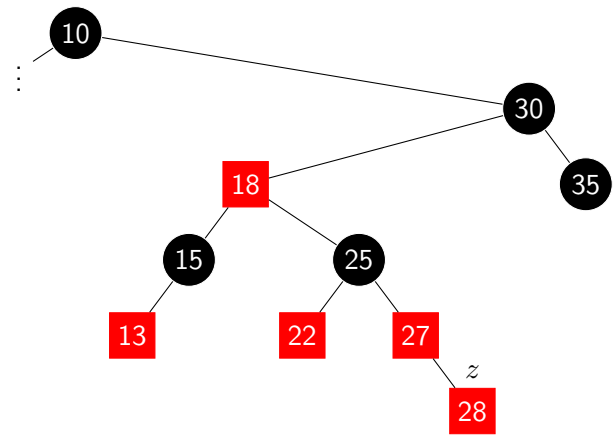


The running time of this method is in $\Theta(h) + \Theta(1) + \Theta(1) = \Theta(h)$. Therefore, the running time of `RBTreeInsert` is $\Theta(h) = \Theta(\log(n))$.

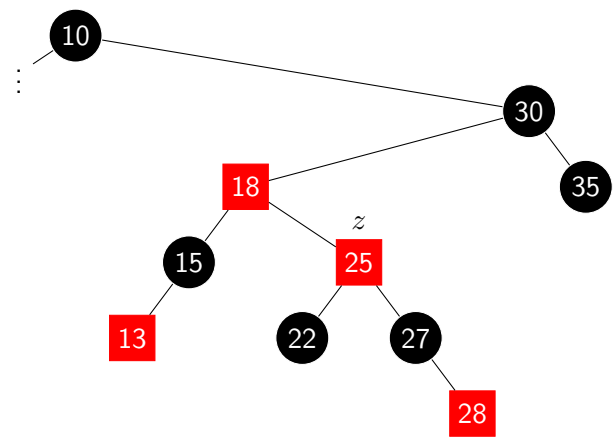
Perform the `RBTreeInsert`(T,28) on the tree given below.



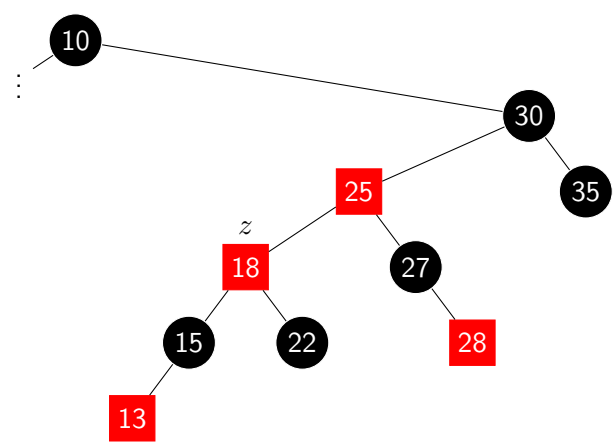
Insert the node into the tree,



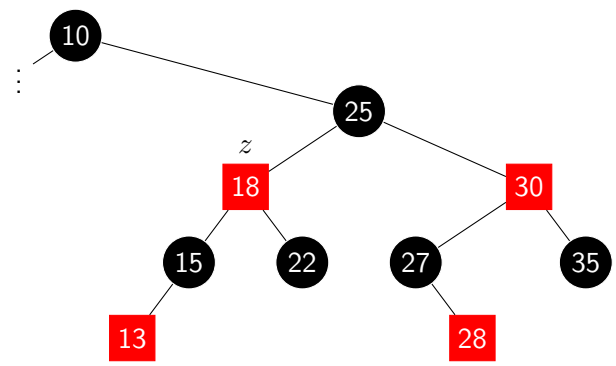
Since the parent (27) and auncle (22) are **red** we must apply Case I. Recolor 22 and 27 to be **black** and recolor 25 to be **red**. Then assign z as the grandparent of 28 (25).



The parent (18) of z is **red**, the auncle (35) is **black**, z is a right child, and z.`parent` is a left child, so we must apply Case II. Left rotate 18 and set z to be 18.



The parent (25) of z is **red**, the auncle (35) is **black**, z is a left child, and z.`parent` is a left child, so we must apply Case III. Right rotate 30, color z.`parent` to be **black** and the sibling of z to be **red**.



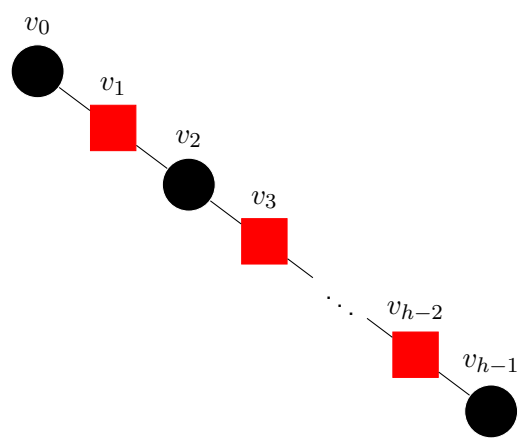
Now, all paths from the root (10) to a leaf contain 4 **black** nodes and every **red** node has two **black** nodes as children.

Appendix

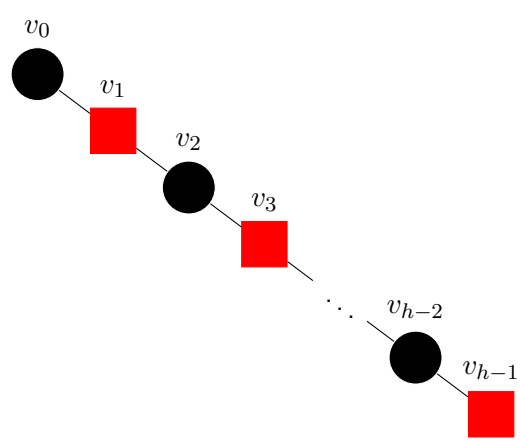
Lemma 1. Suppose T is a "trimmed" **red-black** tree (one where we have removed the **black NIL** nodes) of height $h-1$. The maximum number of **red** nodes in any simple path of length $h-1$ from the root to a leaf is $\lfloor h/2 \rfloor$ and the **minimum** number of black nodes is $\lceil h/2 \rceil$.

Proof. Let $p = \{v_0, v_1, v_2, \ldots, v_{h-1}\}$ be a simple path of length $h-1$ from the root of the tree to a leaf. Further suppose that p is such that it has the maximum number of **red** nodes possible. Notice that we cannot have subsequent nodes both **red**, so the maximum number of red nodes possible is obtained when nodes alternate between **black** and **red**. We continue by considering the parity of $h-1$;

If $h-1$ is even: Then nodes $v_0, v_2, v_4, \ldots, v_{h-1}$ are all **black** (since the root is always **black**) and nodes $v_1, v_3, v_5, \ldots, v_{h-2}$ are all **red**. That is, there are $\frac{h-1}{2}$ **red** nodes and $\frac{h+1}{2}$ **black** nodes.



If $h-1$ is odd: Then nodes $v_0, v_2, v_4, \ldots, v_{h-2}$ are all **black** (since the root is always **black**) and nodes $v_1, v_3, v_5, \ldots, v_{h-1}$ are all **red**. That is, there are $\frac{h}{2}$ **red** nodes and $\frac{h}{2}$ **black** nodes.



Now, we observe that

$$\lfloor h/2 \rfloor = \begin{cases} \frac{h-1}{2} & \text{if } h-1 \text{ is even} \\ \frac{h}{2} & \text{if } h-1 \text{ is odd} \end{cases} \tag{1}$$

and

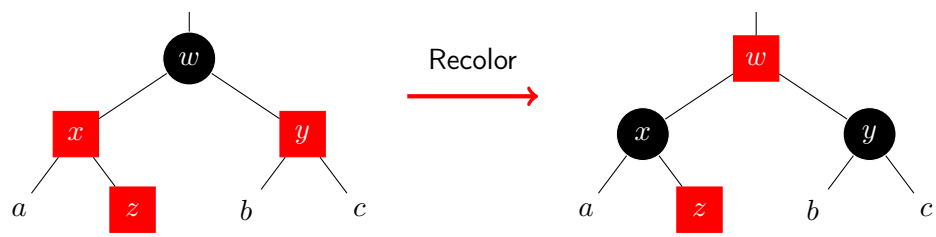
$$\lceil h/2 \rceil = \begin{cases} \frac{h+1}{2} & \text{if } h-1 \text{ is even} \\ \frac{h}{2} & \text{if } h-1 \text{ is odd} \end{cases}. \tag{2}$$

Therefore, the maximum number of **red** nodes in any simple path from the root to a leaf of length $h-1$ is $\lfloor h/2 \rfloor$ and the minimum number of **black** nodes is $\lceil h/2 \rceil$. $\square$

Summary of Cases

Case I If the parent and "auncle" of z are **red**

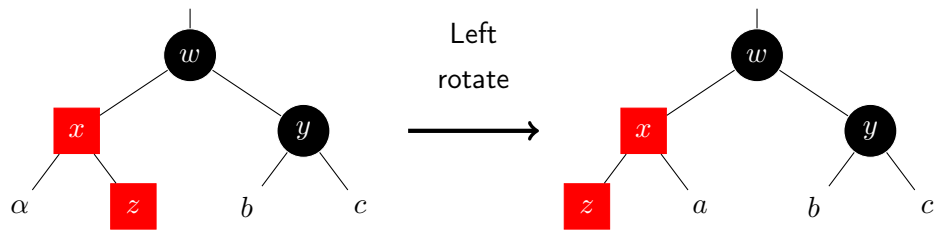
- Color the parent z **black**.
- Color the auncle of z **black**.
- Color the grandparent of z **red**.
- Repeat on the grandparent of z.



function `RBInsertFixupA`(T, **alters** z)
  while ($z \neq T.\texttt{root}$) **and** ($z.\texttt{parent.color} \neq$ **black**) **do**
    $y = \texttt{Sibling}(z.\texttt{parent})$
    if $y.\texttt{color} =$ **black then**
      return
    end if
    $z.\texttt{parent.color} =$ **black**
    $y.\texttt{color} =$ **black**
    $z = z.\texttt{parent.parent}$
    $z.\texttt{color} =$ **red**
  end while
end function

Case II If the parent x of z is **red** and its auncle is **black**. If z is a right child and its parent x is a left child,

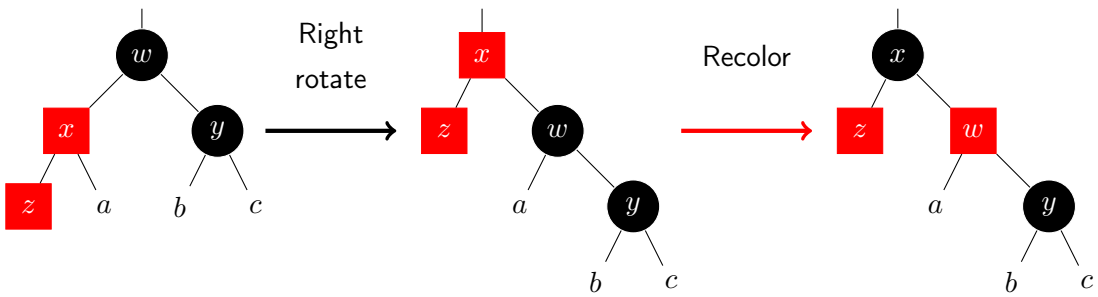
- Left Rotate on x,
- Swap z and x
- Apply algorithm from Case III on z




```
function RBInsertFixupB(T, alters z)
    if (z = T.root) or (z.parent.color = black) then
        return
    end if
    x = z.parent
    w = x.parent
    if (z = x.right) and (x = w.left) then
        z = x
        LeftRotate(T,x)
    else if (z = x.left) and (x = w.right) then
        z = x
        RightRotate(T,x)
    end if
end function
```

Case III If the parent of z is **red** and its auncle is **black**. If z is a left child and its parent is a left child,

- Right Rotate on the grandparent of z,
- Color the parent of z **black**,
- Color the sibling of z **red**




```
function RBInsertFixupC(T, alters z)
    if (z = T.root) or (z.parent.color = black) then
        return
    end if
    x = z.parent
    w = x.parent
    if (z = x.left) and (x = w.left) then
        RightRotate(T,w)
        x.color = black
        w.color = red
    else if (z = x.right) and (x = w.right) then
        LeftRotate(T,w)
        x.color = black
        w.color = red
    end if
end function
```